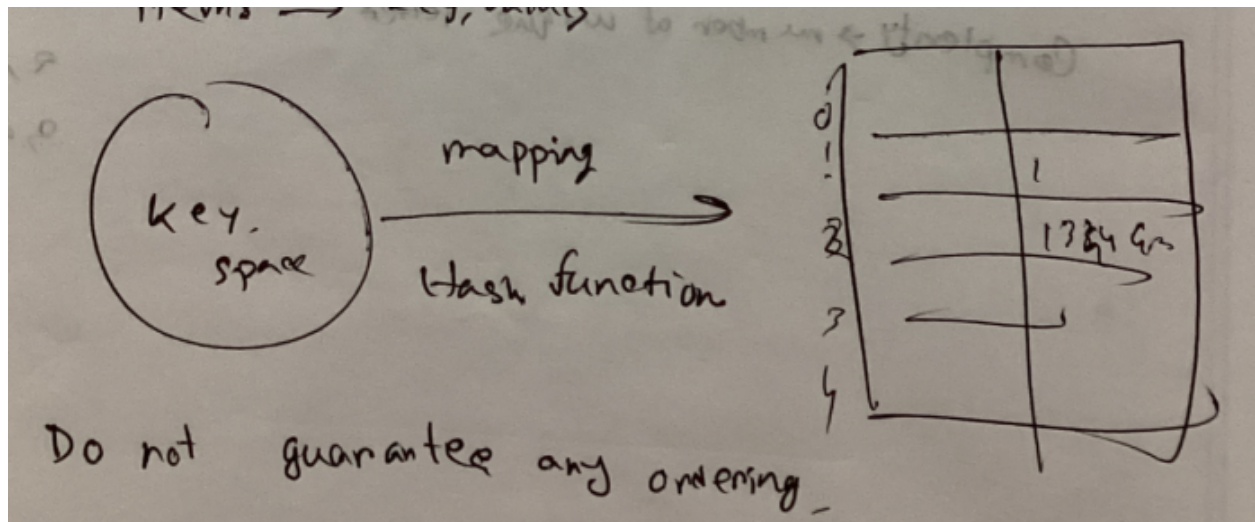


Hashing

Hashing —> is a searching process . In this process a hash function is used to determine the index of a key in an array by giving the key itself as an input to the function. Hashing is the process of mapping key from values.

Hash table —> it is the storage where the values are stored according to their indexes that we get from hashing

Hash Function —> it is the function that takes the key values as the input and provides the index where the key is stored in the hash table.



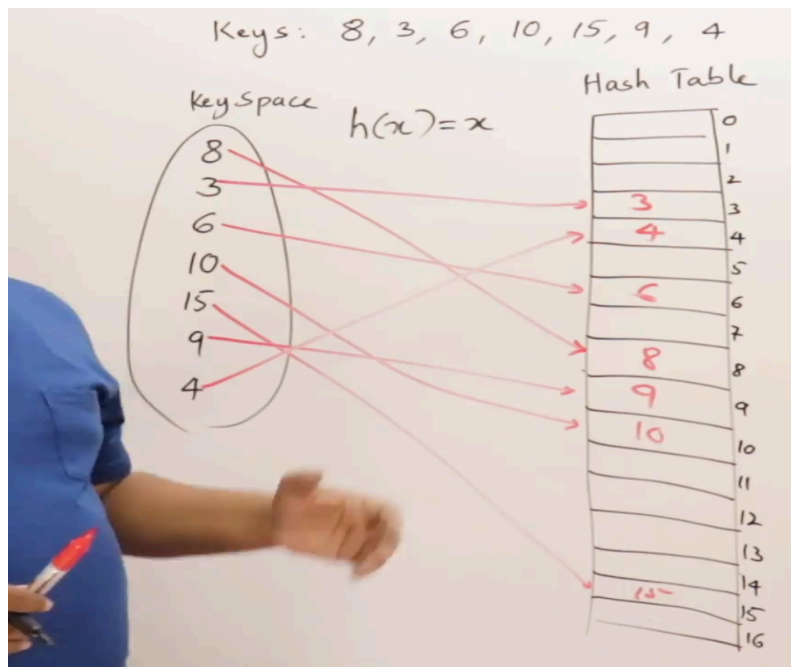
The hash function and mapping works like a normal function.
There are different types of mapping:

Mappings.

- one - one
- one - many
- many - one
- many - many

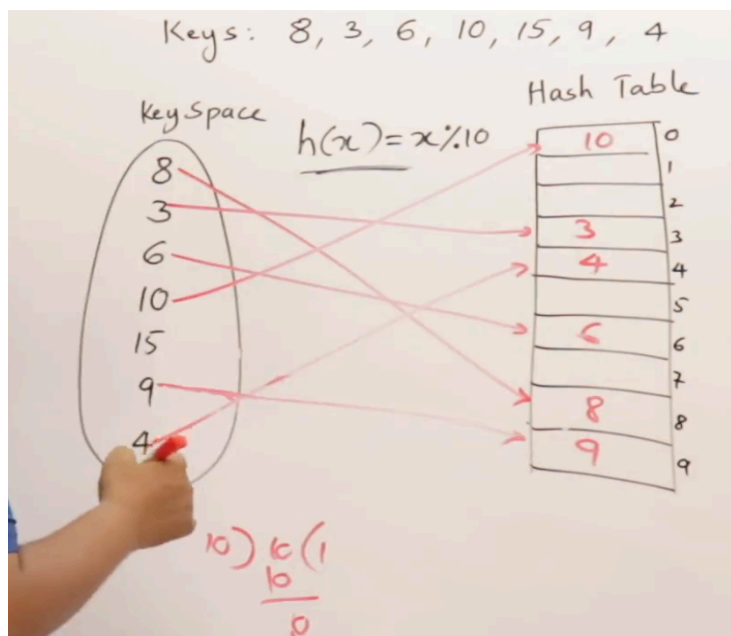
Or mapping, we can consider $h(x)$ as the hash function where x is the element and the result we get from $h(x)$ is the index.

One - one



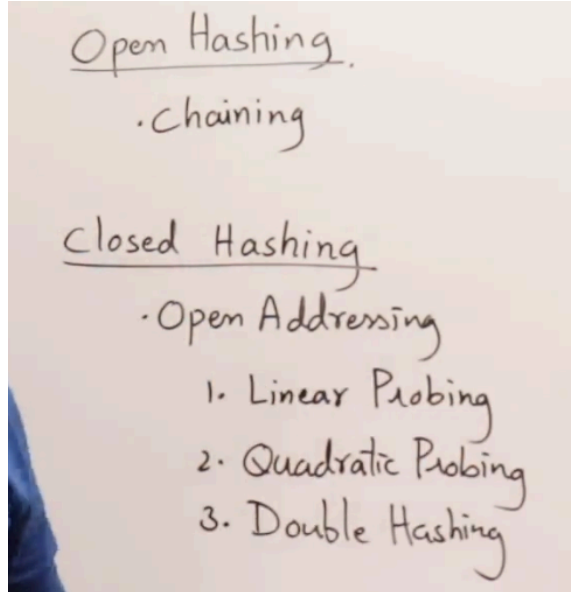
The problem with this is that it takes a lot of extra spaces up to the biggest element

Many -- one



The problem with this is that we can get the same index for more than one element. For example 10 and 20 both will be stored in 0 index. This is called collision.

To resolve collision, there are methods called open hashing and closed hashing.

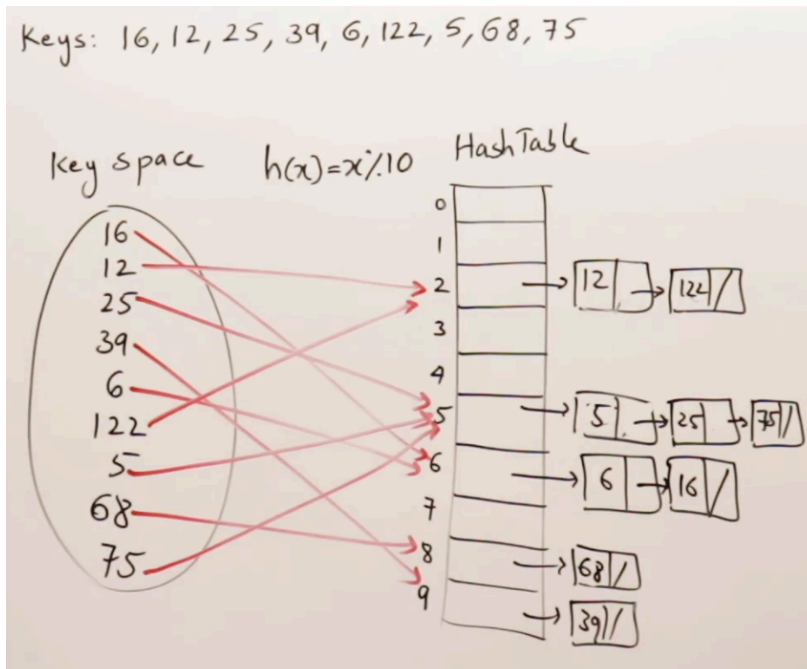


In closed hashing the space used will be equal to the number of elements. And in open hashing the space used will be more than the number of elements.

Chaining

In this method, $H(x) = x \bmod (\text{size})$

Size means the number of indexes in the hash table



In this method, if multiple elements go into the same column, then they form a linked list maintaining their order.

Search:

1. Determine $H(x)$ value for the element
2. Then go to that index and if there is a linked list go through the list.
3. If the key is not found in that linked list then it does not exist among the elements

$$\text{Loading factor} \rightarrow \lambda = \frac{n}{\text{size}}$$

$$n = 100$$

$$\text{size} = 10$$

$$\lambda = \frac{100}{10} = 10$$

n = the number of elements

Size = the number of indexes in the hash table

The complexity calculation here is done by loading factor. It is assumed that there are at most 10 elements in every node or column

$$t = 1 + \lambda$$

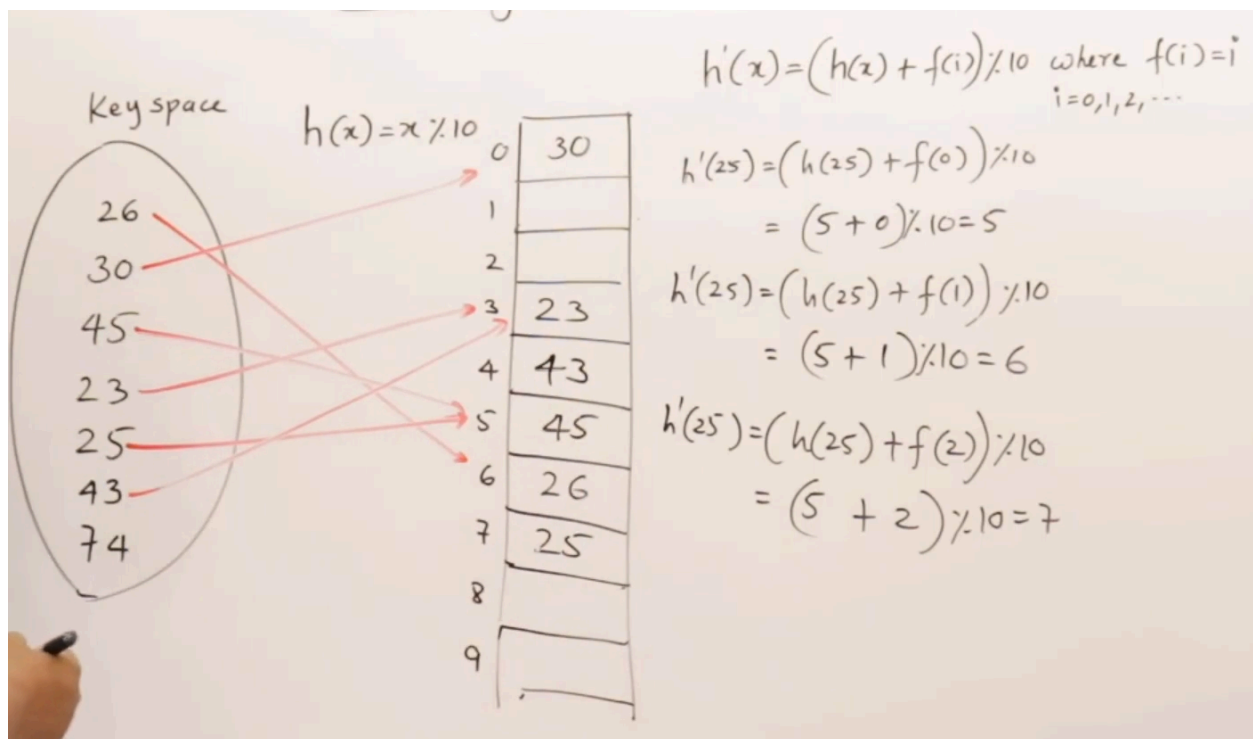
For an unsuccessful search, the time will be

But there is a problem when the elements are like 5, 235, 35, 65, 75, 95 all of these will end up in the same index. We cannot choose chaining in these cases

Linear Probing

$$h(x) = x \% 10$$

$$h'(x) = (h(x) + f(i)) \% 10 \text{ where } f(i) = i \\ i = 0, 1, 2, \dots$$



When there is a collision the modified hash function is used. It basically checks the next index to store the value, and if there is collision also there, it checks the next one and this goes on.

As example here, 25 is initially directed towards 5 index, but there is 45 there. So it checks for 6 index using h' function but there is also 26 there. Then it goes on to next index and it is free so it stores the value there.

Suppose 19 is already stored in 9 index. So the function checks 19 and automatically starts from the 0 index again.

$$\begin{aligned}
 h'(29) &= (h(29) + f(0)) \% 10 \\
 &= (9 + 0) \% 10 = 9 \\
 &= (9 + 1) \% 10 = 0 \\
 &= (9 + 2) \% 10 = 1
 \end{aligned}$$

Quadratic Probing

Hashing Technique (Quadratic Probing)

Key Space $h(x) = x \% 10$

Hash Table

0	
1	
2	
3	23
4	43
5	
6	
7	13
8	27
9	

$h'(x) = (h(x) + f(i)) \% 10$ where $f(i) = i^2$
 $i = 0, 1, 2, \dots$

$h'(13) = (h(13) + f(0)) \% 10$
 $(3 + 0) \% 10 = 3$
 $(3 + 1) \% 10 = 4$
 $(3 + 4) \% 10 = 7$

$7 + 0$
 $7 + 1$
 $7 + 4$
 $7 + 9$

Avg. successful Search
 $\frac{-\log_e(1-\lambda)}{\lambda}$
Avg. unsuccessful Search
 $\frac{1}{1-\lambda}$

This is the same as linear probing just it is $f(i) = i^2$ for this case. All the rest are the same.

Double Hashing

Hashing Technique (Double Hashing)

$$h_1(x) = x \% 10$$
$$h_2(x) = R - (x \% R)$$
$$h'(x) = (h_1(x) + i * h_2(x)) \% 10 \text{ where } i = 0, 1, 2, \dots$$

H.T

Here, R is the prime number that is smaller and nearest to the size of the hash table.

If the size of the table is 10, then R will be 7.

If size of table is 15, R will be 13.

Hashing Technique (Double Hashing)

Key Space $h_1(x) = x \% 10$

H.T

0	
1	15
2	35
3	
4	
5	5
6	
7	
8	25
9	

$h_1(x) = x \% 10$
 $h_2(x) = 7 - (x \% 7)$
 $h'(x) = (h_1(x) + i * h_2(x)) \% 10$ where $i=0,1,2,\dots$

$h'(25) = (5 + 1 * 3) \% 10 = 8$
 $7 - (25 \% 7)$
 $7 - 4 = 3$
 $h'(15) = (5 + 1 * 6) \% 10 = 1$
 $7 - (15 \% 7)$
 $7 - 1 = 6$
 $h'(35) = (5 + 1 * 7) \% 10 = 2$
 $7 - (35 \% 7)$
 $7 - 0 = 7$

Hashing Technique (Double Hashing)

Key Space $h_1(x) = x \% 10$

H.T

0	
1	15
2	35
3	
4	95
5	5
6	
7	
8	25
9	

$h_1(x) = x \% 10$
 $h_2(x) = 7 - (x \% 7)$
 $h'(x) = (h_1(x) + i * h_2(x)) \% 10$ where $i=0,1,2,\dots$

$h'(25) = (5 + 1 * 3) \% 10 = 8$
 $7 - (25 \% 7)$
 $7 - 4 = 3$
 $h'(15) = (5 + 1 * 6) \% 10 = 1$
 $7 - (15 \% 7)$
 $7 - 1 = 6$
 $h'(35) = (5 + 1 * 7) \% 10 = 2$
 $7 - (35 \% 7)$
 $7 - 0 = 7$
 $h'(85) = (5 + 3 * 3) \% 10 = 4$
 $7 - (85 \% 7)$
 $7 - 4 = 3$

